CYBERCLASS INFOTEP

Técnico en Ciberseguridad

Participante: Juan Carlos Montero

Tecnico: Cyberseguridad

Nombre del Módulo: FUNDAMENTOS DE CIBERSEGURIDAD

Facilitador: Kevin Feliz Henríquez

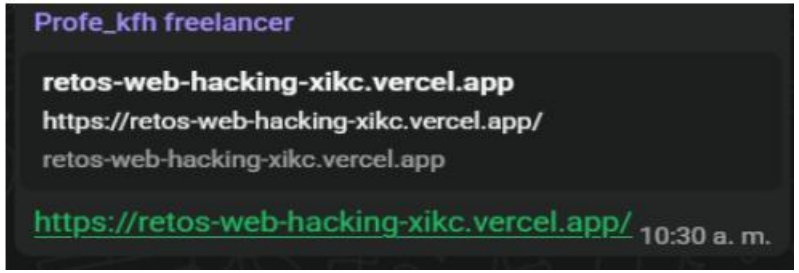
Centro: Cyberclass

Nombre Actividad: Reto 03: LocalStorage Manipulation

Descripción: El reto consiste en la elevación de privilegios y la manipulación de la base de datos local para obtener el acceso

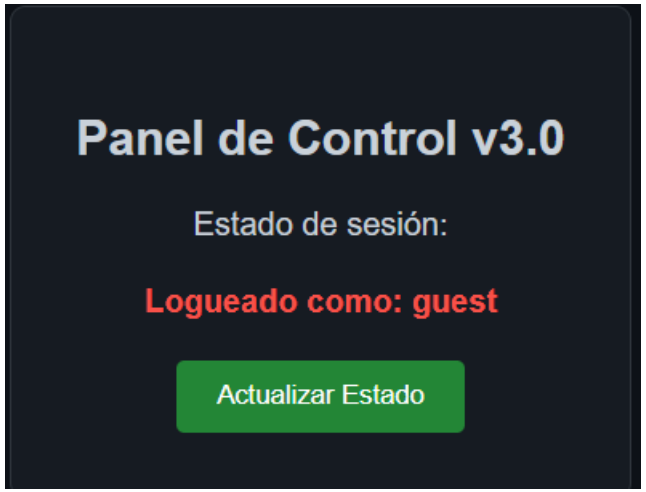
Paso #1

Abrimos el link y acccedemos al reto #3



Paso #2

Vemos que nos indica que estamos logueados como invitado y no temenos acceso por lo cual devemos loguearnos como **admin**



Paso #3

Damos click en **F12** para ver el codigo Fuente y analizarlo.

```
<html lang="es">  view-source  scroll
  <head> ... </head>
  <body>  flex
    <div class="box">
      <h2>Panel de Control v3.0</h2>
      <p>Estado de sesión:</p>
      <div id="status" class="status" style="color: rgb(248, 81, 73);">Logueado como: guest</div> == $0
      <div id="flag-section" class="admin-content" style="display: none;">
        <strong>FLAG:</strong>
        " FLAG{LOCAL_STORAGE_MASTER_2026} "
      </div>
      <button onclick="verificarAcceso()">Actualizar Estado</button>
    </div>
    <script>
            // Inicializar si no existe
            if (!localStorage.getItem('user_role')) {
                localStorage.setItem('user_role', 'guest')
            }

            const verificarAcceso = () => {
                const role = localStorage.getItem('user_role')
                const statusDiv = document.getElementById('status')
                const flagDiv = document.getElementById('flag-section')

                statusDiv.innerText = `Logueado como: ${role}`

                if (role === 'admin') {
                    statusDiv.style.color = '#3fb950'
                    flagDiv.style.display = 'block'
                } else {
                    statusDiv.style.color = '#f85149'
                    flagDiv.style.display = 'none'
                }
            }
```

Paso #4

Luego de analizar el codigo y con ayuda del buscador vemos que la constante (role) se esta guardando en una base de datos local.

1. ¿Qué es `localStorage`?

Es un objeto de la **Web Storage API** que permite guardar datos en el navegador de forma **persistente.** A diferencia de las cookies (que son pequeñas) o del `sessionStorage` (que se borra al cerrar la pestaña), los datos en `localStorage` no tienen fecha de expiración; se quedan ahí aunque cierres el navegador o apagues la computadora.

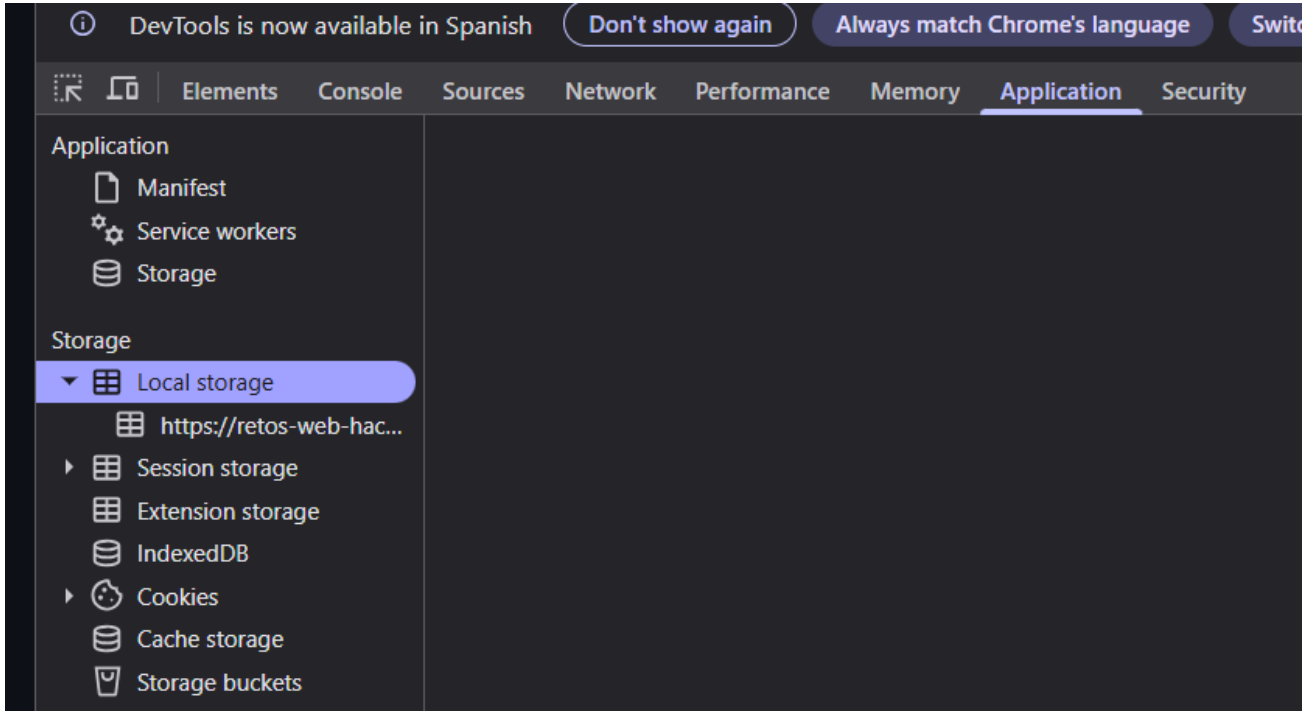
¿Cómo funciona en la práctica?

Imagina que cuando el usuario inició sesión, guardaste su rol (por ejemplo, si es "admin" o "cliente"). Para recuperarlo y saber qué mostrarle en pantalla, usas esa línea:

Acción	
Guardar el dato	`localStorage.setItem('role', 'admin');`

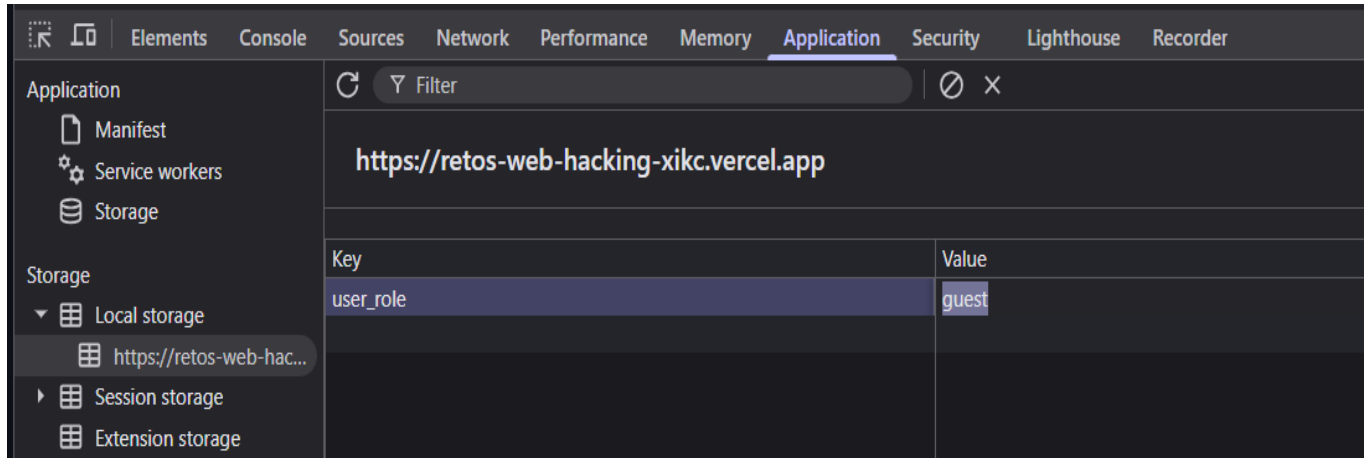
Paso #5

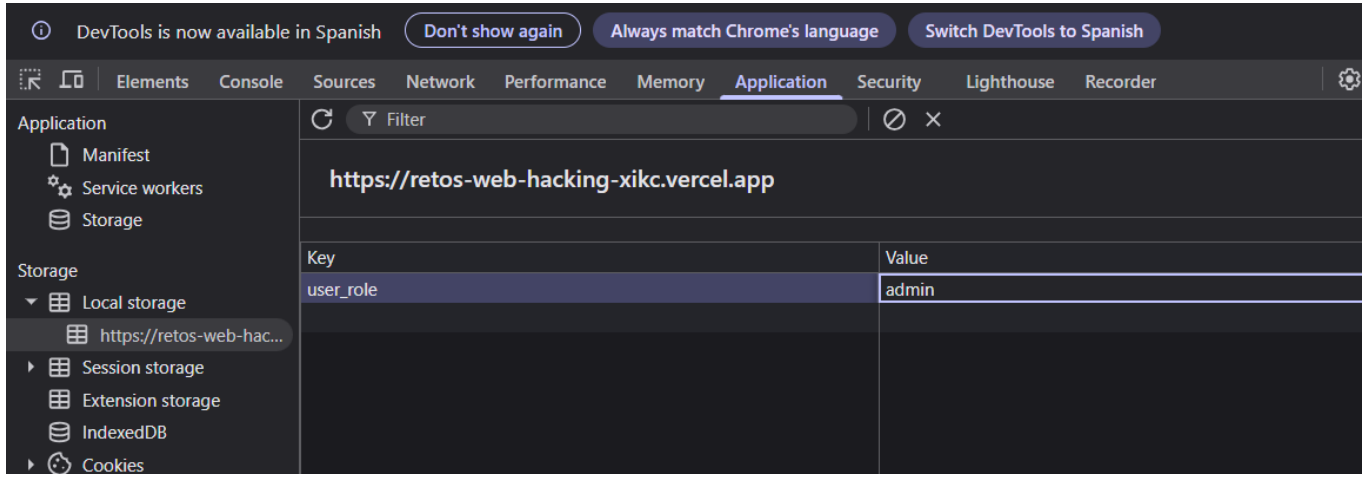
Luego de investigar localizamos **(localStorage**) en la pestaña de Aplicación y como unico accedemos al link de retos-web

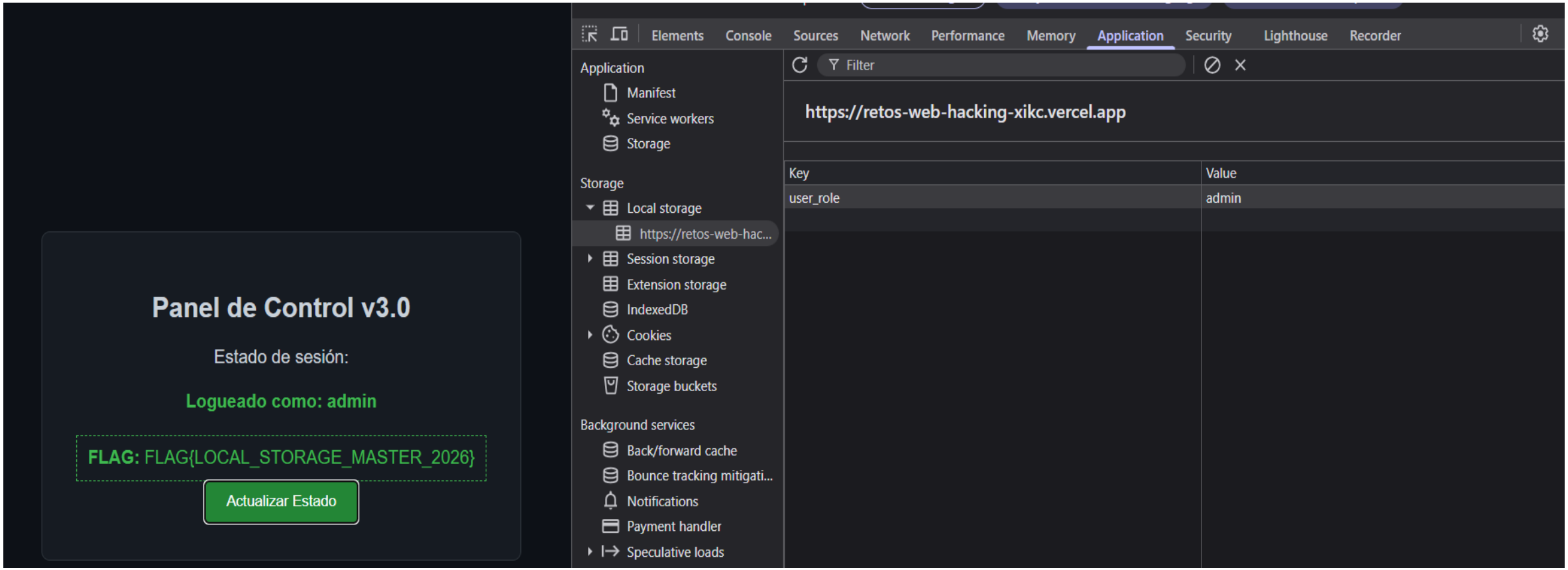


Paso #6

Luego de haber accedido solo cambiamos **guest** por (**admin**) y damos click en actualizar estado






Panel de Control v3.0
Estado de sesión:
Logueado como: admin
FLAG: FLAG{LOCAL_STORAGE_MASTER_2026}
Actualizar Estado
Elements
Console
Sources
Network
Performance
Memory
Application
Security
Lighthouse
Recorder
Application
Manifest
Service workers
Storage
Storage
Local storage
https://retos-web-hac...
Session storage
Extension storage
IndexedDB
Cookies
Cache storage
Storage buckets
Background services
Back/forward cache
Bounce tracking mitigati...
Notifications
Payment handler
Speculative loads
Filter
https://retos-web-hacking-xikc.vercel.app
Key
Value
user_role
admin

Medidas a tomar para evitar la vulnerabilidad

1. Mantener las sesiones y los roles exclusivamente en el servidor.
2. El servidor debe Validar el rol del usuario en cada petición que requiera privilegios especiales.
3. La escalada de privilegios debe requerir autenticación y autorización explíxitas verificadas en el servidor.
4. No exponer datos sensibles.

Resumen de lo aprendido:

En esta activida aprendí:

1. Las validaciones **son mejores** en el servidor.
2. Los roles y sesiones sensibles deben **gestionarse** en el **backend**.
3. El uso de la **herramienta de desarrollo** (pestaña **Application**).
4. La vulnerabilidad de **Control de Acceso Quebrado (Broken Access Control).**